

Chapter 5

Remote Procedure Call & Remote Method Invocation

Outline

5.1. Overview of RPC & RMI

5.2. Stub and skeleton

5.3. The RMI Registry

5.4. The Remote Interface

5.5. Implementing RMI

5.1. Overview of RPC & RMI

- RPC (Remote Procedure Call) and RMI (Remote Method Invocation) are both mechanisms for invoking functions or methods located on remote computers. Here is an overview of each:
- RPC:
 - RPC is a generic concept that refers to any mechanism for making a procedure call across a network.
 - In RPC, the client sends a request to the server to execute a procedure or function, and the server returns the result to the client.
 - RPC allows clients to invoke functions on a remote server as if they were local functions.
 - RPC can be used with a variety of programming languages and protocols, such as XML-RPC, JSON-RPC, gRPC, and CORBA.

5.1. Overview of RPC & RMI ...

RMI

- RMI:
 - RMI is a specific implementation of RPC in the Java programming language.
 - RMI allows Java objects running in one JVM to invoke methods on Java objects running in another JVM, located on a remote machine.
 - RMI uses Java serialization to pass parameters and return values between the client and server, and it provides automatic garbage collection and exception handling.
 - RMI provides a simple and intuitive way to create distributed applications in Java.
- Both RPC and RMI provide a way to develop distributed applications by allowing clients to invoke functions on a remote server. However, RMI is specifically designed for Java applications, while RPC can be used with any programming language or protocol.

RMI

- Remote Method Invocation (RMI) in Java is a powerful mechanism that enables communication between Java objects running on different Java Virtual Machines (JVMs), potentially across different physical machines.
- It allows a client to invoke methods on an object residing on a remote server as if it were a local object.
- RMI is part of Java's core API and is built to facilitate distributed computing in a purely Java-based environment.



How RMI Works

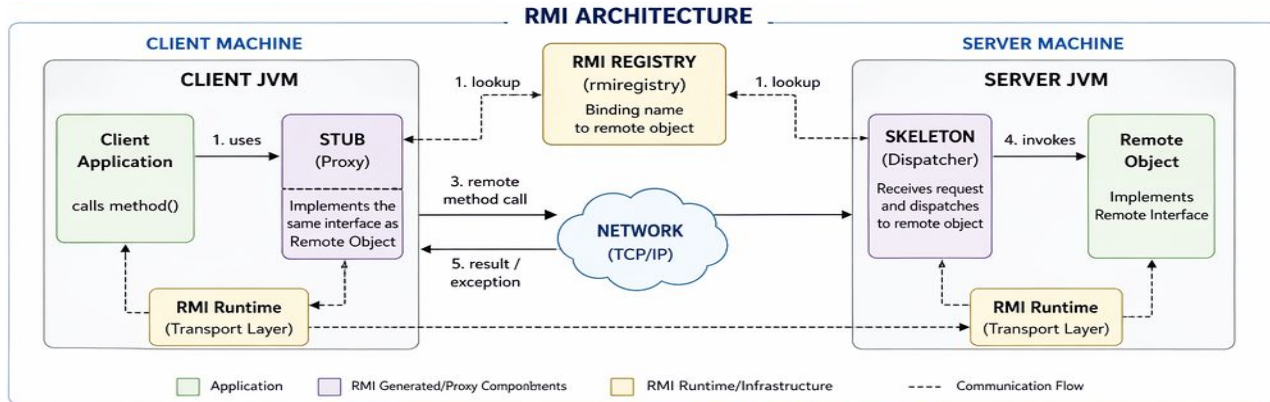
HOW IT WORKS (IN SIMPLE TERMS)

- 1 The client looks up the remote object in the RMI Registry.
- 2 The client gets a reference (stub) to the remote object.
- 3 The client calls a method on the stub.
- 4 The stub sends the request to the remote JVM.
- 5 The remote object executes the method and returns the result, which is sent back to the client.

- Here's a high-level overview of how RMI facilitates remote communication:
- **Server-Side:**
 - The server creates a remote object (an instance of a class implementing a remote interface).
 - The server registers the remote object with the RMI registry under a unique name.
 - The RMI system generates a stub for the remote object, which is used for communication.
- **Client-Side:**
 - The client looks up the remote object in the RMI registry using its name and the server's hostname/port.
 - The registry returns a stub (proxy) for the remote object to the client.
 - The client invokes methods on the stub as if it were the actual remote object.
- **Communication:**
 - The stub serializes the method arguments and sends them to the server over the network.
 - The server's skeleton (or equivalent) receives the request, deserializes the arguments, and invokes the method on the actual remote object.
 - The result (or exception) is serialized and sent back to the client via the stub.
- **Error Handling:**
 - Network failures, remote object unavailability, or serialization issues result in a `RemoteException`, which the client must handle.

RMI Architecture

The RMI architecture consists of several layers and components that work together to enable remote communication. Let's explore each part:



● a. Client and Server

- **Client:** The program or object that invokes a method on a remote object.
- **Server:** The program or object that hosts the remote object and makes it available for clients to access.

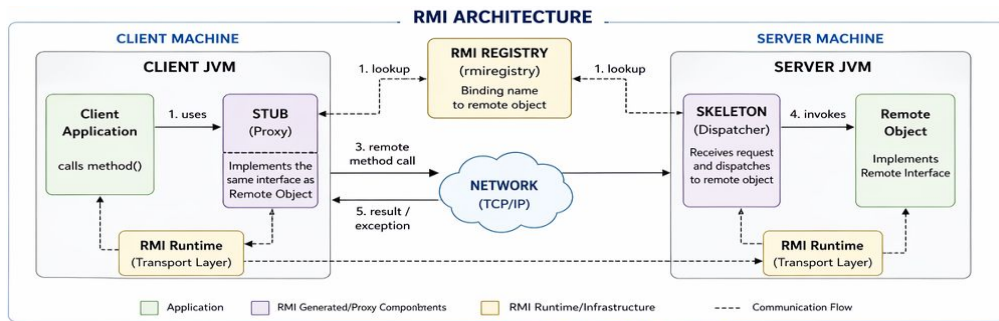
● b. Remote Object

- A **remote object** is an instance of a class that implements a **remote interface**. It lives on the server and provides methods that clients can invoke remotely.
- The remote object must be registered with the RMI registry so clients can locate it.

● c. Remote Interface

- A **remote interface** is a Java interface that declares the methods that can be invoked remotely. It extends the `java.rmi.Remote` interface.
- Each method in the remote interface must throw `java.rmi.RemoteException` to handle potential network-related errors.

RMI Architecture



● d. Stub and Skeleton

- **Stub:** A client-side proxy object that represents the remote object. The client calls methods on the stub, which forwards the calls to the server over the network.
- **Skeleton:** A server-side helper that receives the remote method calls, unmarshals the parameters, and invokes the actual method on the remote object. **(Note: In modern Java versions, skeletons are automatically generated and not explicitly created by developers.)**
- **Stubs and skeletons handle the marshalling (serializing objects into a byte stream) and unmarshalling (deserializing byte streams back into objects) of method parameters and return values.**

● e. RMI Registry

- The **RMI registry** is a naming service that runs on the server and maps remote objects to names (or keys). Clients use the registry to look up remote objects by name.
- **The registry typically runs on a well-known port (default: 1099) and is implemented by the `rmiregistry` command.**

● f. Transport Layer

- The transport layer handles the low-level network communication between the client and server, using TCP/IP sockets.
- RMI abstracts this layer, so developers don't need to manage sockets directly.

5.2. Stub and skeleton

Stub

- In the context of Remote Procedure Call (RPC) and (RMI), stub and skeleton are important components that help in the communication between the client and the server.
- A stub is a client-side proxy that represents the remote object and provides a local interface for the client to interact with the remote object.
- When the client invokes a method on the stub, the stub marshals the parameters, sends the request to the server, waits for the response, and unmarshals the result before returning it to the client.
- The stub hides the details of the remote communication and makes it appear as if the remote object is a local object.
- In RMI, the stub is generated automatically by the RMI compiler and is responsible for handling the communication with the remote server.

skeleton

- A skeleton is a server-side proxy that receives requests from the client, unmarshals the parameters, invokes the remote method, marshals the result, and sends it back to the client.
- The skeleton is responsible for handling the low-level details of the remote communication and providing a transparent interface to the server object.
- In RMI, the skeleton is generated automatically by the RMI runtime system.
- In summary, the stub is a client-side proxy that communicates with the server, while the skeleton is a server-side proxy that communicates with the client.
- Both stub and skeleton are generated automatically by the RMI compiler/runtime and are responsible for hiding the low-level details of the remote communication and providing a transparent interface to the client and server, respectively.

cont.

STUB (on Client Side)



The Stub is a client-side proxy for the remote object. It has the same methods as the remote interface.

When the client calls a method on the stub:

- It marshals (serializes) the method name and arguments.
- Sends the request to the server (to the Skeleton) over the network.
- Waits for the result, unmarshals (deserializes) it.
- Returns the result to the client.

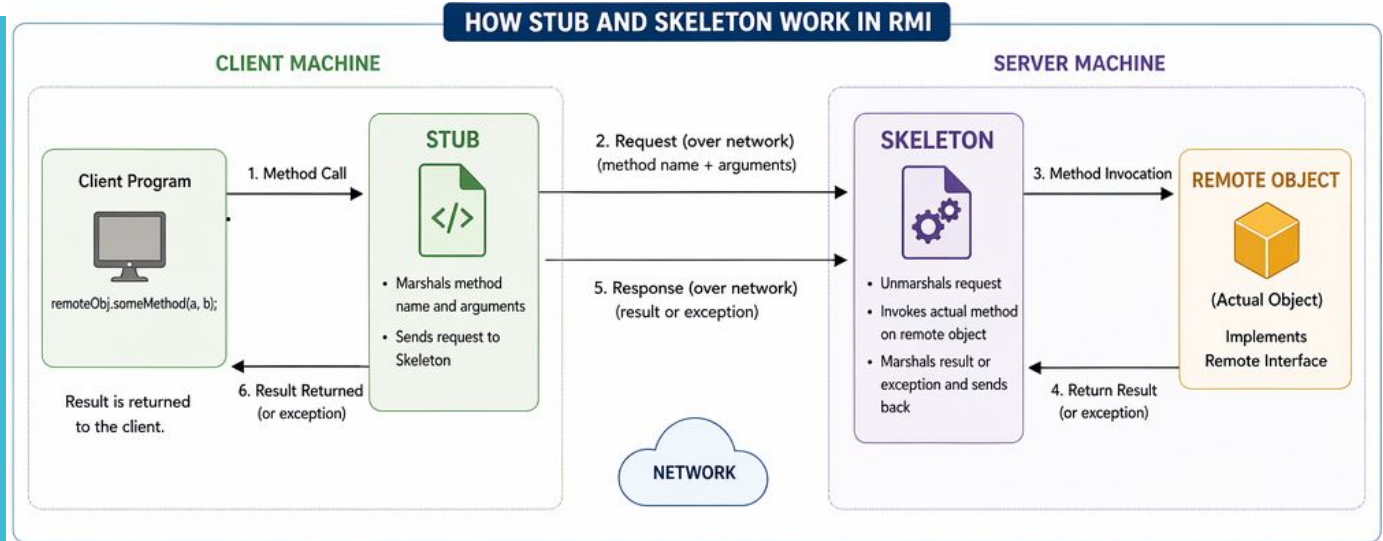
SKELETON (on Server Side)



The Skeleton is a server-side proxy for the remote object. When it receives a request from the stub:

- It unmarshals the method name and arguments.
- Invokes the actual method on the real remote object.
- Marshals the return value (or exception).
- Sends the response back to the stub over the network.

Cont...



FLOW EXPLAINED STEP-BY-STEP

- 1 The client calls a method on the remote object reference.
- 2 The Stub receives the call, serializes (marshals) the method name and arguments, and sends the request to the server-side Skeleton.
- 3 The Skeleton receives the request, deserializes (unmarshals) the data, and invokes the actual method on the remote object.
- 4 The remote object executes the method and returns the result (or throws an exception) to the Skeleton.
- 5 The Skeleton marshals the result/exception and sends it back to the Stub.
- 6 The Stub unmarshals the result/exception and returns it to the client.

5.3. The RMI Registry

- The RMI Registry is a central component of Remote Method Invocation (RMI) in Java.
- It is a simple naming service that allows clients to **look up remote objects by their names and obtain references to those objects for remote method invocation.**
- **The RMI Registry serves as a central repository for storing references to remote objects.**
- When a server creates a remote object, it registers the object with the RMI Registry using a name that identifies the object.
- The name can be any string that uniquely identifies the object within the registry.
- Clients can then look up the remote object in the RMI Registry by its name, and the registry returns a reference to the object. The client can then use the reference to invoke methods on the remote object as if it were a local object.

5.3. The RMI Registry ...

- The RMI Registry can be started as a separate process or can be started within the server process itself.
- When the registry is started as a separate process, it listens on a specific port for incoming requests from clients.
- The client can then connect to the registry using the host name and port number of the server machine.
- The RMI Registry is a lightweight and simple naming service that provides an easy way for clients to obtain references to remote objects in RMI-based distributed applications.
- It plays a key role in enabling RMI clients to interact with remote objects without needing to know their physical location or network details.

5.4. The Remote Interface

- In Java, the Remote Interface is an interface that enables objects to be accessed remotely, across different Java Virtual Machines (JVMs) or even across different machines.
- Remote Interfaces are used in distributed applications where different parts of the application are running on different machines and need to communicate with each other.
- Remote Interfaces in Java are implemented using the Java Remote Method Invocation (RMI) framework, which provides a mechanism for invoking methods on objects running in different JVMs.
- **To create a Remote Interface in Java, you need to define an interface that extends the `java.rmi.Remote` interface and declare the methods that can be invoked remotely.** For example:

5.4. The Remote Interface ...

interface keyword
No method bodies here —
only the method signatures
that must be implemented.

```
import java.rmi.Remote;  
import java.rmi.RemoteException;
```

```
public interface MyRemoteInterface extends Remote {  
    public void doSomething() throws RemoteException;  
}
```

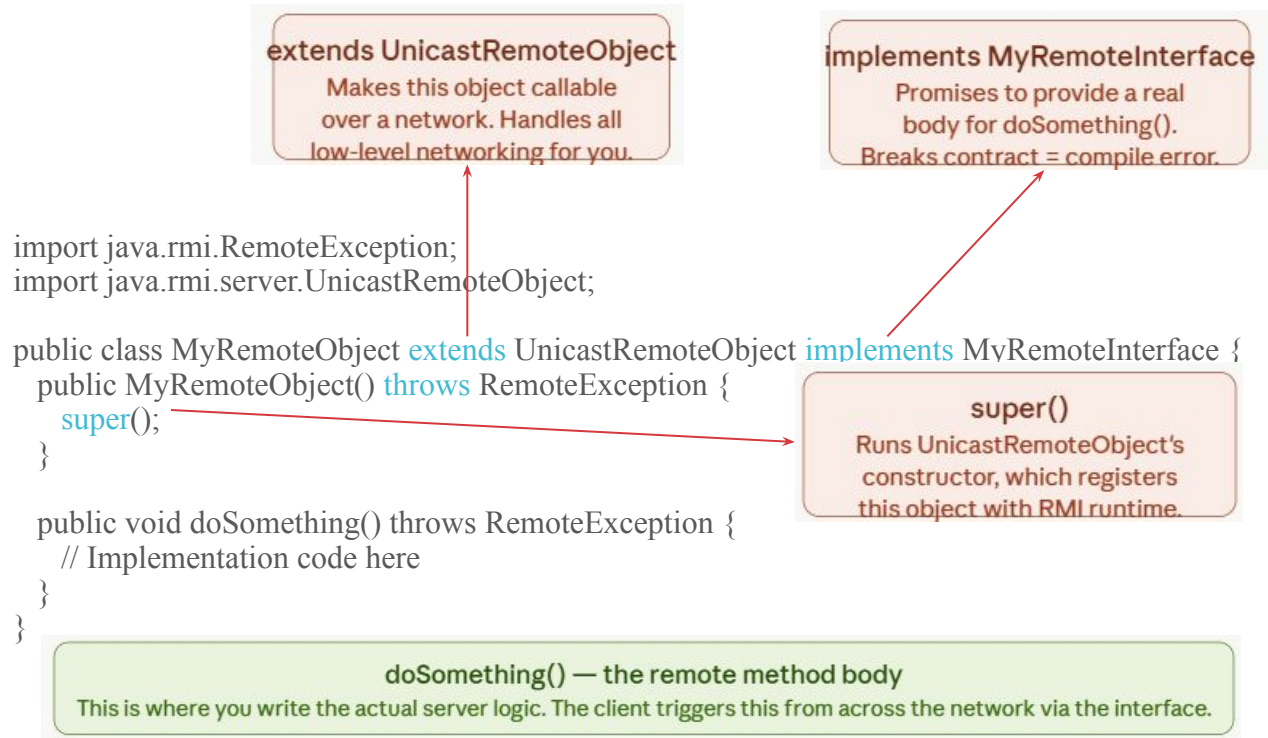
extends Remote
Marks this as a special
RMI-capable interface.
Required by Java RMI rules.

throws RemoteException
Every method in the interface
must declare this — network
calls can always fail.

Once you have defined the Remote Interface, you need to implement it on the server-side and make it available for remote access.

This is typically done by creating a remote object that implements the Remote Interface and registering it with the RMI registry. For example:

5.4. The Remote Interface ...



`java.rmi.Naming`

Provides `rebind()` to register objects in the RMI registry by name

`LocateRegistry`

A utility class that can start a registry inside the current JVM process

`new MyRemoteObject()`

Calls the constructor which runs `super()` — this automatically exports the object to the network. No extra step needed.

Why `MyRemoteObject` type?

We use the concrete class here (not the interface) so we can pass it to `rebind()`. The registry stores the full object.

`java.rmi.registry.LocateRegistry`

Lets you start or find an RMI registry programmatically inside your JVM

`createRegistry(1099)`

1099 is the default RMI port. The registry is like a phone book for objects

Alternative approach

You can also run `rmiregistry` in terminal instead of this line

`rebind()` vs `bind()`

`bind()` throws an error if the name already exists. `rebind()` overwrites it. Always use `rebind()` during development.

URL breakdown

`rmi://` → protocol
`localhost` → server address
`MyRemoteObject` → lookup name

```
import java.rmi.Naming;  
import java.rmi.registry.LocateRegistry;
```

```
public class MyRemoteServer {  
    public static void main(String[] args) {  
        try {  
            // Step 1: Start the RMI registry programmatically on port 1099 (default)  
            LocateRegistry.createRegistry(1099);  
            System.out.println("RMI Registry started on port 1099...");
```

```
            // Step 2: Create an instance of the remote object  
            MyRemoteObject remoteObject = new MyRemoteObject();  
            System.out.println("Remote object created...");
```

```
            // Step 3: Bind the remote object to a name in the registry  
            // Clients will use this exact name in Naming.lookup()  
            Naming.rebind("rmi://localhost/MyRemoteObject", remoteObject);  
            System.out.println("Remote object bound as 'MyRemoteObject'.");  
            System.out.println("Server is ready and waiting for client calls...");
```

```
        } catch (Exception e) {  
            System.err.println("Server error: " + e.getMessage());  
            e.printStackTrace();  
        }  
    }  
}
```

On the client-side, you can then access the remote object by looking it up in the RMI registry and invoking its methods as if it were a local object.

The RMI framework takes care of the details of marshalling and unmarshalling parameters and return values across the network. For example:

5.4. The Remote Interface

...

```
import java.rmi.Naming;  
import java.rmi.RemoteException;  
  
public class MyRemoteClient {  
    public static void main(String[] args) {  
        try {  
            MyRemoteInterface remoteObject = (MyRemoteInterface) Naming.lookup("rmi://localhost/MyRemoteObject");  
            remoteObject.doSomething();  
        } catch (Exception e) {  
            e.printStackTrace();  
        }  
    }  
}
```

Step 2 — cast to interface

The registry returns a generic Object. We cast it to MyRemoteInterface so we can call its methods.

Step 1 — Naming.lookup()

Contacts the RMI registry running on localhost and asks for an object named "MyRemoteObject".

Step 3 — obj.doSomething()

Looks like a normal method call — but it actually travels over the network to run on the server!

① **Server**
creates &
binds object

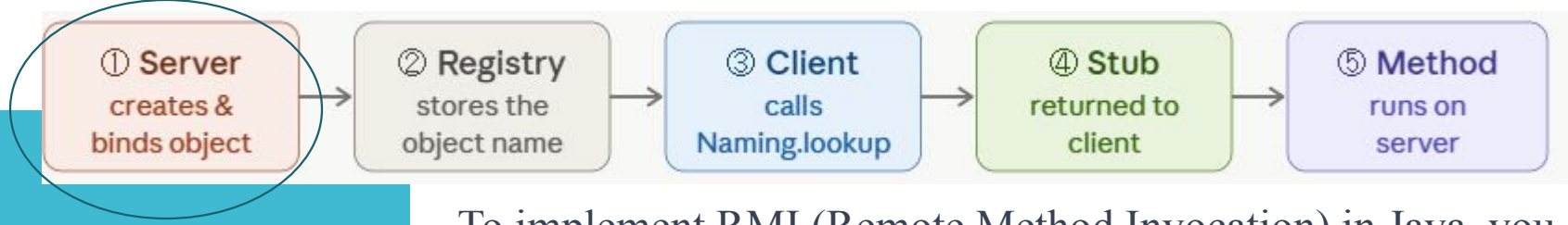
② **Registry**
stores the
object name

③ **Client**
calls
Naming.lookup

④ **Stub**
returned to
client

⑤ **Method**
runs on
server

5.5. Implementing RMI



To implement RMI (Remote Method Invocation) in Java, you need to follow the following steps:

- 1. Define the Remote Interface:** The first step is to define the remote interface that specifies the methods that can be invoked remotely.

The interface should extend the **java.rmi.Remote** interface and each method should declare **java.rmi.RemoteException**. For example:

```
import java.rmi.Remote;
import java.rmi.RemoteException;

public interface MyRemoteInterface extends Remote {
    public String sayHello(String name) throws RemoteException;
}
```

5.5. Implementing RMI ...



2. Implement the Remote Interface: The next step is to implement the remote interface in a class that provides the actual implementation of the methods. This class should extend the **java.rmi.server.UnicastRemoteObject** class and have a constructor that declares **java.rmi.RemoteException**.

3. For example:

```
import java.rmi.RemoteException;
import java.rmi.server.UnicastRemoteObject;
public class MyRemoteImpl extends UnicastRemoteObject implements
MyRemoteInterface {
    public MyRemoteImpl() throws RemoteException {
        super();
    }
    public String sayHello(String name) throws RemoteException {
        return "Hello, " + name + "!";
    }
}
```

5.5 Implementing RMI ...

Create and Start the RMI Registry: The RMI registry is a simple naming service that allows clients to look up remote objects by name. To create and start the



RMI registry, you can use the **java.rmi.registry.LocateRegistry** class. For example:

```
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

public class MyRemoteServer {
    public static void main(String[] args) {
        try {
            Registry registry = LocateRegistry.createRegistry(1099);
            MyRemoteImpl remoteImpl = new MyRemoteImpl();
            registry.bind("MyRemote", remoteImpl);
            System.out.println("Remote object bound to registry.");
        } catch (Exception e) {
            System.err.println("Error: " + e.getMessage());
            e.printStackTrace();
        }
    }
}
```

5.5. Implementing RMI ...



Create the Client: Finally, you can create the client that accesses the remote object by looking it up in the RMI registry using the **java.rmi.Naming** class. For example:

```
import java.rmi.Naming;
```

```
import java.rmi.RemoteException;
```

```
public class MyRemoteClient {
```

```
    public static void main(String[] args) {
```

```
        try {
```

```
            MyRemoteInterface remoteObject = (MyRemoteInterface) Naming.lookup("rmi://localhost/MyRemote");
```

```
            String result = remoteObject.sayHello("Alice");
```

```
            System.out.println(result);
```

```
        } catch (Exception e) {
```

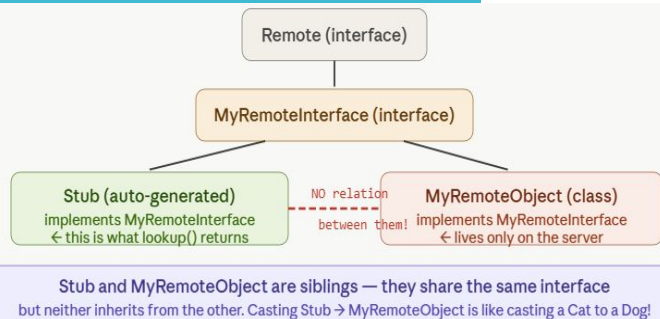
```
            System.err.println("Error: " + e.getMessage());
```

```
            e.printStackTrace();
```

```
        }
```

```
    }
```

```
}
```



Naming.lookup() returns a **stub** (a proxy object),
not the actual MyRemoteObject. The stub implements
MyRemoteInterface, so casting to the interface is safe and correct.

5.5. Implementing RMI

When you run the server and client, the output should be:

Remote object bound to registry.

Hello, Alice!